LLM Inference & Speculative Decoding

Technical Reference

Topics covered

Autoregressive generation · Forward passes · Memory bandwidth bottleneck · Tokens and embeddings · Q, K, V attention · Causal masking · KV cache · Vocabulary scoring · Speculative decoding · Rejection sampling

Table of Contents

1. Inference vs Training

Two distinct phases in a model's life:

- **Training** = the slow, expensive process of teaching the model. Done once. Adjusts billions of parameters by showing the model trillions of words and using gradient descent. Like a student studying for years.
- **Inference** = using the trained model to generate output. Done every time someone asks a question. Like a student taking the exam.

Every time you hit "send" in ChatGPT, that is an inference call. This document is entirely about inference.

2. How LLMs generate text — one token at a time

The single most important fact most people miss:

> *An LLM does NOT generate a whole sentence at once. It generates ONE TOKEN at a time.*

If you ask "What is the capital of France?", ChatGPT does not produce "The capital of France is Paris." in one shot. It produces:

Step	Output so far
1	The
2	The capital
3	The capital of
4	The capital of France
5	The capital of France is
6	The capital of France is Paris
7	The capital of France is Paris.

Each step generates exactly one new token by running the entire model from start to finish. This is called autoregressive generation — "auto" (self) + "regressive" (looking back). The model uses its own previous outputs as input for the next prediction.

If you want 100 tokens of output, the model runs 100 times.

3. What is a forward pass?

A forward pass is one full trip through the model from input to output. Think of it as an assembly line:

```
input tokens → Layer 1 → Layer 2 → ... → Layer 80 → output: probability for next
token
```

Each layer transforms the data using its share of the model's parameters. At the end, the model produces a probability distribution over the entire vocabulary, and we pick one token from that distribution.

> *One forward pass = one token generated.*

For Llama-3-70B (a model with 80 layers and 70 billion parameters):

- One forward pass takes roughly 50 ms on an H100 GPU.
- Generating 100 tokens of output takes about 5 seconds.

4. Why LLM inference is slow — the memory bottleneck

This is the surprising fact that motivates everything else (including speculative decoding).

The model is just a giant pile of numbers

A neural network is fundamentally a collection of floating-point numbers called weights or parameters. These were tuned during training and are now fixed.

Model	Number of parameters
GPT-2 small	117 million
Llama-3-8B	8 billion
Llama-3-70B	70 billion
GPT-4 (rumored)	~1.7 trillion

Each number takes:

- 4 bytes in FP32 (32-bit float)
- **2 bytes in FP16 (half precision)** — most common for inference
- 1 byte in INT8 (quantized)

So Llama-3-70B in FP16 is $70 \times 10^9 \times 2 = 140$ GB of weights.

The memory hierarchy

Where weights live	Capacity	Speed
Disk (SSD)	Terabytes	~5 GB/s (very slow)
GPU VRAM (HBM)	80 GB on H100	~3 TB/s (fast)
GPU compute units	Tiny scratch	~30 TB/s effective for math

GPU compute is roughly 10× faster than VRAM. The compute units are starving for data — they finish their math faster than memory can supply the next batch of weights.

The kitchen analogy

Imagine a chef (GPU compute) who chops vegetables incredibly fast, but the pantry (VRAM) is in the next room.

- Walking to the pantry to fetch ingredients takes 47 seconds.
- Chopping the ingredients takes 5 seconds.

The chef spends most of their time walking, not cooking.

This is exactly LLM inference. Loading the 140 GB of weights from VRAM into the compute units takes ~47 ms. The math itself takes ~5 ms. Loading dominates.

The crucial insight that enables speculative decoding

> *Loading weights to process 1 token takes the same time as loading them to process 5 tokens.*

If you walked to the pantry once and grabbed enough ingredients for 5 dishes, the walking time is the same. The chopping/cooking scales a tiny bit, but the bottleneck (the walking) doesn't.

Same for the GPU: once the weights are loaded, you can process multiple tokens in parallel for almost the same cost as one. This is being memory-bandwidth-bound.

But generation is sequential — you need token 4 to predict token 5. You can't naively process them in parallel… unless you can guess future tokens in advance. That is the speculative decoding insight, covered in Section 13.

5. Tokens, embeddings, and the model's view of language

Tokens

The model does not see English words. It sees integer IDs. A tokenizer chops your text into pieces and assigns each piece an ID:

Text	Token ID (illustrative)
"what"	1820
" is"	374
" the"	279
" capital"	6864
" of"	315
" france"	49141

Note: spaces are part of tokens in modern tokenizers. A "token" can be:

- A whole word ("paris", "the")
- A subword ("un", "able", "##ing")
- A punctuation mark (".", "?")
- A code symbol ("{", "==", "   ")
- A single byte for non-Latin scripts

A typical vocabulary is ~128,000 tokens.

Embeddings — IDs become meaning vectors

The model stores a giant embedding matrix of shape (vocab_size, embedding_dim). For Llama-3 that is (128000, 4096). One row per possible token, each row a 4096-dimensional vector of learned numbers.

When the input arrives, each token ID is replaced by its embedding row. The token "france" becomes a vector like [0.71, 0.38, -0.15, 0.62, ...] — 4096 numbers that encode the meaning of "france."

These embeddings are learned during training. Tokens with similar meanings end up near each other in vector space. "France" and "Germany" are close. "France" and "banana" are far apart.

> *At this stage, no token knows about any other token. "france" looks the same whether the sentence is "France is in Europe" or "I love French food." Context comes next, through attention.*

6. The attention mechanism: Q, K, V

This is the heart of the transformer.

The library analogy (build the intuition)

Imagine walking into a library. You have a question: "I want to learn about World War 2."

Each book has:

- **A title/spine** advertising what it is about ("Cooking", "WW2 History", "Math")
- **Contents** — the actual information inside.

To use the library:

- Compare your question against each book's title, getting a relevance score.
- Read each book's contents weighted by relevance — mostly the WW2 book, a sprinkle of others.
- Combine into your final understanding.

Map this to attention:

- **Query (Q)** = your question. "What am I looking for?" — produced by the current token.
- **Key (K)** = a book's title. "What do I advertise?" — produced by every token.
- **Value (V)** = a book's contents. "What information do I actually deliver?" — produced by every token.

The mechanics

For each token, three weight matrices (W_Q, W_K, W_V) transform the embedding into three different vectors:

```
Q = embedding · W_Q   ("what I'm asking")
K = embedding · W_K   ("what I offer")
V = embedding · W_V   ("what I deliver")
```

Same input vector, three different transformations, three different roles.

Computing attention output for one token

Take a token's Q. Compare it (dot product) against every other token's K. The result is a similarity score per token. Apply softmax to turn the scores into percentages that sum to 1. Use those percentages to mix the V vectors together.

```
score_i = Q · K_i                         for each token i
weights = softmax(score_1, score_2, ...)
attention_output = Σ weights_i · V_i
```

That is the entire attention computation. The current token's output is a weighted blend of every token's V, where the blend weights come from how well each token's K matches the current token's Q.

Concrete example: narrating Q, K, and V

Use the sentence: "The cat sat on the mat. It was tired."

Goal: when the model processes the word "It", it must figure out what "It" refers to. Doing so means computing attention for "It". Q, K, and V each play a different role. Read each block below as if every token is publicly broadcasting one of three distinct messages.

Q (Query) — what each token is asking for

Q is the question the current token poses to the rest of the sequence. It is generated only when that token's turn comes to compute attention. Different tokens ask very different questions.

If we narrated each token's Q vector for the same sentence, it would sound like:

- Q_The = "I am a determiner. I am looking for a noun to introduce."
- Q_cat = "I am the subject. I am looking for a verb that describes my action."
- Q_sat = "I am a verb. I am looking for a place or object related to where the sitting happened."
- Q_on = "I am a preposition. I am looking for an object I can attach to."
- Q_mat = "I am the object. I am looking for the determiner that introduces me."
- **Q_It = "I am a pronoun. I am looking for a noun I might refer to."**
- Q_was = "I am a linking verb. I am looking for a subject and a state."
- Q_tired = "I am an adjective. I am looking for the subject I describe."

These Q's drive the search. Each one is short-lived — used once for that token's attention, then discarded. The most important takeaway: every token has its own Q, and different tokens are looking for very different things.

K (Key) — what each token advertises about itself

K is what every token publishes for others to read — its "title and spine." Unlike Q, the K of a token is computed once and then read repeatedly by every future token's Q during attention. Narrated K vectors for our sentence:

- K_The = "I am a determiner." → low relevance to most queries
- K_cat = "I am a noun, a small animal, a typical sentence subject." → **high relevance to Q_It (which seeks a noun)**
- K_sat = "I am a verb describing a posture." → low relevance to Q_It
- K_on = "I am a preposition introducing a location." → low
- K_mat = "I am a noun, an object on the floor." → medium relevance to Q_It (also a noun, but less likely the antecedent of "It")

When Q_It compares itself against each of these K's via dot product, it scores K_cat highest. After softmax, the attention weights might be:

```
[K_The: 0.05, K_cat: 0.70, K_sat: 0.05, K_on: 0.05, K_mat: 0.10, K_It: 0.05]
```

So K is the matchmaker. It is what makes Q_It "land" on cat instead of mat or sat.

V (Value) — what each token actually delivers if attended to

V is the content payload. Once the attention weights are decided (via Q and K), the V's get mixed using those weights. V is what gets passed forward into the rest of the layer. Narrated:

- V_The = "My contribution: nothing semantically rich, just a determiner marker."

- **V_cat = "My contribution: the meaning of cat — a small furry mammal, the subject of this sentence, alive, capable of being tired."**
- V_sat = "My contribution: the action of sitting, past tense, completed."
- V_on = "My contribution: a positional relationship."
- V_mat = "My contribution: the meaning of mat — a flat object on the floor, inanimate."
- V_It = "My contribution: pronoun-ness, awaiting resolution."

Mixing using the weights from above:

```
new_It = 0.05·V_The + 0.70·V_cat + 0.05·V_sat
       + 0.05·V_on + 0.10·V_mat + 0.05·V_It
```

The result is dominated by V_cat. So after this attention step, the token "It" is no longer just a vague pronoun. Its updated representation now carries: small furry mammal, the subject, alive, can be tired. That is the resolution of "It" → "cat", expressed in vectors.

Side-by-side summary for the same sentence

Token	Q (what I ask)	K (what I advertise)	V (what I deliver)
The	Looking for a noun to introduce	I am a determiner	Determiner marker
cat	Looking for a verb of action	I am a noun, a small animal, subject	Furry mammal, alive, can be tired
sat	Looking for a location	I am a verb of posture	Sitting action, past tense
on	Looking for an object to attach to	I am a preposition	Positional relation
mat	Looking for my determiner	I am a noun, an inanimate object	Flat object on floor, inanimate
It	Looking for a noun to refer to	I am a pronoun (vague)	Pronoun-ness, awaiting resolution
was	Looking for subject and state	I am a linking verb	Linking verb, past tense
tired	Looking for what I describe	I am an adjective	Tiredness, a state

> *Q, K, V are not three things. They are three different views of the same token, each computed via its own learned matrix, each playing its own role: ask, advertise, deliver.*

Three key facts about Q, K, V

- **They are three transformations of the same input.** Each token's embedding produces its own Q, K, and V at every layer.
- **Q, K, V change every layer.** Each of the 80 layers has its own W_Q, W_K, W_V matrices, so the same token has 80 different Q's across the network.

- **Q is consumed once, K and V are consumed many times.** The current token's Q does its dot product, gets the answer, and is never used again. K and V get read by every future token. This is the basis of caching.

7. Causal masking — the property that enables caching

In autoregressive language models (GPT, Llama, Claude), attention has one crucial restriction:

> *A token can only attend to itself and previous tokens. NEVER to future tokens.*

This is called causal masking or autoregressive masking.

Why?

During training, the model learns to predict the next word. If token 1 could see token 5, it would already know what comes after token 4 — that would be cheating. So we restrict attention to a triangular pattern.

The attention pattern as a triangle

For 4 tokens, the attention table looks like this. ✓ = "this query attends to this key", ✗ = "blocked by mask":

	K_1	K_2	K_3	K_4
Q_1	✓	✗	✗	✗
Q_2	✓	✓	✗	✗
Q_3	✓	✓	✓	✗
Q_4	✓	✓	✓	✓

- Token 1 only attends to itself.
- Token 2 attends to tokens 1, 2.
- Token 3 attends to tokens 1, 2, 3.
- Token 4 attends to tokens 1, 2, 3, 4.

What happens when token 5 arrives?

	K_1	K_2	K_3	K_4	K_5
Q_1	✓	✗	✗	✗	✗
Q_2	✓	✓	✗	✗	✗
Q_3	✓	✓	✓	✗	✗
Q_4	✓	✓	✓	✓	✗
Q_5	✓	✓	✓	✓	✓

Look carefully at rows 1–4. Nothing changed. Token 1 still attends only to K_1. Token 2 still attends to K_1 and K_2. The mask blocks past tokens from looking at future tokens, so adding a future token does not affect their attention output.

Only the new row (Q_5) does new work, attending to K_1 through K_5.

> *This is the fundamental reason the KV cache works.*

8. The KV cache — what it is and why it works

The wasteful problem

When generating the second token, all the work the model did on the prompt during the first forward pass would be repeated from scratch. Embeddings, Q/K/V computations, attention — all redone. For a 1000-token prompt, that is enormously wasteful.

The cache

Because of causal masking, K and V vectors of past tokens never change once computed. So we save them.

The KV cache stores, at every layer:

- All past K vectors
- All past V vectors

When generating token N+1:

- Look at cache: it already has K_1..K_N and V_1..V_N from past forward passes.
- Compute Q, K, V for the new token only (cheap — one token's worth of work).
- **Append** K_new and V_new to the cache. Cache now has K_1..K_{N+1} and V_1..V_{N+1}.
- Use the cache for attention: the new token's Q dots against all K's in cache, mixes all V's in cache.
- Continue forward pass.

Crucial properties of the cache

- **Append-only.** Old entries never change. K_1 stays as K_1 forever.
- **Read by every future token.** K_1 gets read when generating token 2, again for token 3, again for token 1000. That is why we cache it — it is consumed many times.
- **Q is not cached.** The current token's Q is used once (for its own attention), then discarded. Only K and V get cached because they need to be read again by future tokens.
- **Stored per layer.** All 80 layers each maintain their own K and V cache. So per token, we cache 2 vectors × 80 layers × 4096 dims × 2 bytes ≈ 1.3 MB. For 100K tokens, that is ~130 GB — which is why long-context KV cache compression is its own active research area.

KV cache does not save weight loading

Important nuance: the KV cache saves attention recomputation but does NOT save weight loading. We still need to push the new token's vector through all 80 layers, multiplying by all the weight matrices. That weight load is still ~50 ms per token.

> *The cache trims redundant attention work, not the bottleneck.*

9. Worked example with a 20-word vocabulary

Let us shrink everything to toy size and trace one full forward pass with real numbers.

Setup

Vocabulary (20 tokens):

ID	Word	ID	Word
0	what	10	london
1	is	11	berlin
2	the	12	tokyo
3	capital	13	a
4	of	14	city
5	france	15	country
6	germany	16	in
7	japan	17	europe
8	india	18	.
9	paris	19	<EOS>

Embedding dimension: 4 (real models use 4096; we use 4 to keep arithmetic visible).
Layers: 1 (real models have 80; we use 1 for clarity).

Embedding matrix E (shape 20 × 4)

```
ID  Word        d0    d1    d2    d3
0   what       0.1   0.2  -0.1   0.3
1   is         0.0   0.5   0.3   0.1
2   the       -0.2   0.1   0.4   0.0
3   capital    0.6  -0.1   0.2   0.5
4   of         0.1   0.0  -0.3   0.2
5   france     0.7   0.3   0.5  -0.1
6   germany    0.6   0.4   0.4  -0.2
7   japan      0.5   0.3   0.6  -0.1
8   india      0.4   0.2   0.5   0.0
9   paris      0.8   0.4   0.6  -0.2
10  london     0.7   0.5   0.5  -0.3
11  berlin     0.6   0.4   0.5  -0.2
12  tokyo      0.7   0.4   0.5  -0.2
13  a         -0.1   0.0   0.1   0.2
14  city       0.3   0.2   0.1   0.4
15  country    0.2   0.3   0.2   0.5
16  in         0.0   0.1  -0.1   0.2
```

```
17  europe     0.4   0.5   0.3   0.1
18  .         -0.3  -0.3  -0.3  -0.3
19  <EOS>     -0.5  -0.5  -0.5  -0.5
```

Notice how related words have similar vectors: capital cities (paris, london, berlin, tokyo) cluster together. Countries cluster together. This is what the model learns during training.

Step 1: Tokenize and embed

Prompt: "what is the capital of france" → token IDs [0, 1, 2, 3, 4, 5].

Look up each ID in E. Stack into matrix X (shape 6 × 4):

```
          d0    d1    d2    d3
what     0.1   0.2  -0.1   0.3
is       0.0   0.5   0.3   0.1
the     -0.2   0.1   0.4   0.0
cap.     0.6  -0.1   0.2   0.5
of       0.1   0.0  -0.3   0.2
france   0.7   0.3   0.5  -0.1
```

Step 2: Compute Q, K, V

Three weight matrices live in the layer (each 4 × 4). For simplicity we use identity matrices, so Q = K = V = X. (In real models these are different, learned matrices.)

Each of the 6 tokens has a Q, K, V vector — all 4-dimensional.

Step 3: Attention for the LAST token (france)

We only need a prediction for what comes after "france", so we compute attention for token 6 only.

Q_6 = [0.7, 0.3, 0.5, -0.1] (france's query).

Compute $Q_6 \cdot K_i$ for every i (dot products):

i	Token	K_i	$Q_6 \cdot K_i$
1	what	[0.1, 0.2, -0.1, 0.3]	0.07 + 0.06 - 0.05 - 0.03 = 0.05
2	is	[0.0, 0.5, 0.3, 0.1]	0 + 0.15 + 0.15 - 0.01 = 0.29
3	the	[-0.2, 0.1, 0.4, 0.0]	-0.14 + 0.03 + 0.20 + 0 = 0.09
4	capital	[0.6, -0.1, 0.2, 0.5]	0.42 - 0.03 + 0.10 - 0.05 = 0.44
5	of	[0.1, 0.0, -0.3, 0.2]	0.07 + 0 - 0.15 - 0.02 = -0.10
6	france	[0.7, 0.3, 0.5, -0.1]	0.49 + 0.09 + 0.25 + 0.01 = 0.84

Raw scores: [0.05, 0.29, 0.09, 0.44, -0.10, 0.84].

Apply softmax. e^score: [1.05, 1.34, 1.09, 1.55, 0.90, 2.32]. Sum = 8.25. Divide:

Token	Attention weight
what	0.13
is	0.16
the	0.13
capital	0.19
of	0.11
france	0.28

Sums to 1. ✓

Now mix the V vectors:

```
new_x_france = 0.13·V_what + 0.16·V_is + 0.13·V_the
             + 0.19·V_capital + 0.11·V_of + 0.28·V_france
```

Computing each component and summing:

Dim	Calculation	Sum
d0	0.013 + 0.000 - 0.026 + 0.114 + 0.011 + 0.196	0.308
d1	0.026 + 0.080 + 0.013 - 0.019 + 0.000 + 0.084	0.184
d2	-0.013 + 0.048 + 0.052 + 0.038 - 0.033 + 0.140	0.232
d3	0.039 + 0.016 + 0.000 + 0.095 + 0.022 - 0.028	0.144

x_france_after_attention = [0.308, 0.184, 0.232, 0.144].

This is "france in context of the prompt." It has been blended with hints from every previous token.

(Real models would also pass through a feed-forward network and 79 more layers. Skipping for clarity.)

Step 4: Project to vocabulary scores (logits)

The output matrix W_out has shape (4, 20). A common trick is weight tying — set W_out = E.transpose(). So W_out[d, j] = E[j, d].

Compute logits = x_france_after_attention · W_out, producing 20 scores:

j	Word	logit
0	what	0.088
1	is	0.176

j	Word	logit
2	the	0.049
3	capital	0.285
4	of	-0.010
5	france	0.373
6	germany	0.323
7	japan	0.334
8	india	0.276
9	paris ← highest	0.430
10	london	0.381
11	berlin	0.346
12	tokyo	0.377
13	a	0.021
14	city	0.210
15	country	0.235
16	in	0.024
17	europe	0.299
18	.	-0.260
19	<EOS>	-0.434

Step 5: Softmax and pick

Apply softmax over all 20 logits to get probabilities. Pick the highest (greedy sampling) → paris (token ID 9).

Important takeaway from this example

> *Every prediction step computes a score for every word in the vocabulary. No shortcuts. For our toy: 20 scores. For Llama-3: 128,000 scores. Every. Single. Token.*

The output projection is one matrix multiplication of shape (1, 4096) · (4096, 128000) = (1, 128000). About 524 million multiplications, all parallelized on the GPU.

Sampling settings (temperature, top-k, top-p) are filters applied after these full logits are computed. They do not skip the all-vocab scoring.

10. Generating a multi-token response

Generation = the loop in Section 9 repeated until the model emits an end-of-sequence token.

Round-by-round trace

Assume a chat-tuned model that produces a full-sentence answer to "what is the capital of france":

Round	Input sequence (last token in CAPS)	Picked token
1	what is the capital of FRANCE	the
2	what is the capital of france THE	capital
3	what is the capital of france the CAPITAL	of
4	what is the capital of france the capital OF	france
5	what is the capital of france the capital of FRANCE	is
6	what is the capital of france the capital of france IS	paris
7	...france is PARIS	.
8	...france is paris .	<EOS>

The user only sees the generated portion: "the capital of france is paris."

What happens in each round

- Take the entire sequence so far.
- Look up the embedding of just the new last token.
- Compute Q, K, V for the new token at every layer.
- Append new K and V to the cache at every layer.
- Run attention: new token's Q dots against all K's in cache.
- Pass through layer's feed-forward, repeat for next layer.
- Final layer's last-token vector → output projection → 128,000 logits → softmax → pick → new token.

Each round is a full forward pass. For an 8-token response, the big model runs 8 forward passes.

Why generation can drift (hallucinations)

There is no checking mechanism. If at round 6 the model accidentally picks "London" instead of "Paris" because of some probability quirk, the response becomes "the capital of france is London." and the model keeps going as if that were a fact. The model doesn't know it made a mistake — it just predicts the next token given everything before.

This is a fundamental property of autoregressive generation. It is why prompt engineering tricks like "Let's think step by step" help — they get the model to commit reasoning to the sequence first, which then conditions the final answer.

11. Output projection and vocabulary scoring

A short consolidation of Section 9's most important step.

The model's last-layer vector for the final token is some 4096-dimensional point. We need to turn that into a probability distribution over vocabulary tokens.

```
logits = final_vector @ W_out         # (1, 4096) @ (4096, 128000) = (1, 128000)
probabilities = softmax(logits)        # 128000 numbers summing to 1
```

W_out has shape (embedding_dim, vocab_size). For Llama-3 that is (4096, 128000) ≈ 524M parameters just for the output projection. Often this matrix is tied to the embedding matrix (set W_out = E.transpose()) to save parameters.

Sampling from the distribution

After softmax, you have probabilities for every vocab token. Several ways to pick the next token:

- **Greedy (temperature = 0):** always pick the highest-probability token. Deterministic but boring.
- **Temperature sampling:** divide logits by T before softmax. Lower T → sharper, more deterministic. Higher T → flatter, more random.
- **Top-k:** keep only the K highest-probability tokens, set rest to -infinity, renormalize, sample.
- **Top-p (nucleus):** keep the smallest set of tokens whose cumulative probability ≥ p, sample from those.

All of these operate after the full 128K logits have been computed.

12. Prompt and response are one sequence

A common confusion: people think "input tokens" and "output tokens" are separate. They are not.

> *From the model's perspective, there is only one sequence of tokens. The prompt is just the part the user wrote. The response is just the part the model wrote. Both occupy the same sequence and the same KV cache.*

Implications:

- **Context window applies to prompt + response combined.** A 128K context window means total length, not 128K each.
- **The model treats its own past outputs as facts.** Once a token is generated, it joins the sequence and influences every subsequent prediction.
- **Few-shot prompting works** because examples in the prompt condition the model the same way past output would.
- **Chain-of-thought prompting works** because reasoning tokens generated by the model become part of the input for predicting the final answer.
- **System prompts** are just tokens at the start of the sequence that get processed first.
- **Multi-turn conversations** are concatenated as one big sequence: [system][user1][assistant1][user2][assistant2]...

13. Speculative decoding — the speedup trick

Now we can finally explain speculative decoding properly.

The setup recap

- One forward pass through Llama-3-70B takes ~50 ms (dominated by weight loading).
- Generating 8 tokens normally takes 8 × 50 = 400 ms.
- A forward pass that processes 4 new tokens takes about the same 50 ms (because weight loading is the bottleneck, not math).

If only we could process multiple new tokens in parallel... we can't, because each token depends on the previous. Unless we guess.

The two-model setup

- **Target model** (the slow, accurate one): Llama-3-70B. Distribution it produces over the vocabulary: p(x).
- **Draft model** (a tiny fast one): Llama-3-1B, ~70× smaller. Distribution: q(x). Its forward pass is ~1 ms.

The algorithm in plain English

For each iteration of the outer loop:

- **Draft phase.** The tiny model autoregressively guesses the next K tokens (say K=4). Total cost: ~4 ms.
- **Verify phase.** The big model runs ONE forward pass on the prompt + the 4 drafted tokens. Because the big model processes all positions in parallel, this single pass produces the big model's predicted distribution at all 4 drafted positions plus one bonus position. Cost: ~50 ms (the same as predicting 1 token).
- **Accept/reject loop.** Walk left to right through the 4 drafted tokens. At each position, decide whether to accept the draft's guess or reject it.
- **Output.** All accepted tokens are kept. If everything is accepted, we get K+1 tokens (including the bonus). If rejection happens, accept everything before it, take a corrected token at the rejection point, throw away the rest.

Concrete walkthrough

Prompt: "what is the capital of france". Big model would naturally generate "the capital of france is paris."

Draft phase (tiny model, ~4 ms):

- Tiny model autoregressively produces 4 tokens: "the", "capital", "of", "france". (These are easy filler tokens; tiny models do well on these.)

Verify phase (big model, ~50 ms):

- Big model takes "what is the capital of france the capital of france" as input — a single forward pass.
- It produces 4 distributions, one at each drafted position.
- Big model says: at position 1 it would have generated "the" ✓, at position 2 "capital" ✓, at position 3 "of" ✓, at position 4 "france" ✓. All match.

Bonus token: The big model also gives a free 5th distribution at the position right after "france", and it predicts "is".

> *Result: 5 tokens accepted ("the", "capital", "of", "france", "is") for ~54 ms. Without speculation, 5 tokens would have cost 250 ms. ~5× speedup.*

Continue iterating. The next round drafts 4 more tokens, verifies, etc.

Why it is lossless — the rejection rule

The draft model's guesses can be wrong. We need to filter them in a way that produces the same distribution as the target model would alone. The rule:

```
For each drafted token x with draft probability q(x) and target probability p(x):
    accept with probability  min(1, p(x) / q(x))

If rejected, sample a corrected token from:
    p'(x) = max(0, p(x) - q(x)) / Z
    where Z = Σ max(0, p(x') - q(x'))
```

You can prove that under this rule, the probability of outputting any particular token x equals exactly p(x) — the target distribution. (Sketch: split into "draft proposed x and was accepted" + "rejection happened and corrected sample is x". Both pieces sum to p(x) regardless of which case we are in.)

> *This is what makes speculative decoding mathematically lossless. The output is statistically indistinguishable from running the big model alone. Only the latency changes.*

Acceptance rate and speedup

If α is the average per-token acceptance rate, expected tokens accepted per iteration is roughly:

```
E[tokens] = (1 - α^(K+1)) / (1 - α)
```

- For α = 0.7 and K = 4: ~3.0 tokens per iteration → ~3× speedup.
- For α = 0.9 and K = 4: ~4.1 tokens per iteration → ~4× speedup.

When it works well vs poorly

- **Works well:** code completion, structured output (JSON), repetitive text, low temperature. Both models agree most of the time.
- **Works poorly:** highly creative text, high temperature, out-of-distribution generation. Models disagree often, low acceptance rate.
- **Worst case:** even if every draft is rejected, you still get 1 corrected token per iteration — same as baseline. You cannot be slower than normal generation.

Variants worth knowing (just names)

- **EAGLE / EAGLE-2:** the "draft" is a tiny extra head bolted onto the target model itself. Higher acceptance rate.
- **Medusa:** multiple prediction heads on the target predicting different future positions in parallel.
- **Lookahead decoding:** no draft model — generates n-grams from previous outputs.

14. Glossary of terms

Term	Meaning
Inference	Running a trained model to produce outputs.
Forward pass	One trip through the entire model from input to output, producing one token.
Token	The atomic unit the model sees. An integer ID corresponding to a word, subword, or symbol.
Vocabulary	The full set of tokens the model knows. ~128,000 for modern LLMs.
Embedding	A vector representation of a token. Looked up from a learned (vocab_size, dim) table.
Weight / parameter	A learned floating-point number inside the model. Llama-3-70B has 70 billion of these.
VRAM	Video RAM, the GPU's main memory. ~80 GB on H100.
Memory bandwidth-bound	The model spends most of its time reading data, not computing.
Layer	One stage of the network. A modern LLM has 30–100+ layers. Each does attention plus a feed-forward network.
Attention	The mechanism by which a token "looks at" other tokens to incorporate context.
Q (Query)	What a token is asking. Computed as embedding · W_Q.
K (Key)	What a token advertises. Computed as embedding · W_K.
V (Value)	What a token delivers if attended to. Computed as embedding · W_V.
Causal mask	The rule that a token can only attend to itself and earlier tokens, never to future ones.
KV cache	A store of past tokens' K and V vectors that lets us avoid recomputing them on every forward pass.
Logits	The raw scores the model assigns to each vocabulary token before softmax.
Softmax	A function that turns a list of numbers into a probability distribution (positive, sums to 1).
Output projection	The matrix that maps the final-layer vector to vocabulary scores. Shape (dim, vocab_size).
Autoregressive	Generation style where each new token is conditioned on all previous tokens.
EOS token	A special "end of sequence" token. When generated, generation stops.
Context window	The maximum sequence length (prompt + response) the model can handle.
Greedy decoding	Always picking the highest-probability token.
Temperature	A factor that scales logits before softmax. Lower → sharper, higher → flatter distribution.
Top-k sampling	Restrict sampling to the K highest-probability tokens.

Term	Meaning
Top-p (nucleus) sampling	Restrict sampling to the smallest set of tokens whose total probability ≥ p.
Speculative decoding	Using a small fast model to guess multiple future tokens, then verifying them in a single forward pass of the big model.
Draft / target model	Small fast model / big accurate model in speculative decoding.
Acceptance rate	Fraction of drafted tokens the big model accepts. Drives the speedup.
Rejection sampling	The math trick that makes speculative decoding lossless — output distribution matches the big model alone.

15. Mental models cheat sheet

A condensed list of the most important mental models from this document.

LLM generation

- One token at a time. Each new token = one forward pass.
- Each forward pass uses every weight in the model.
- Loading weights from VRAM (~50 ms) dominates. Math is much faster.

What the model sees

- A growing sequence of integer IDs, nothing more.
- Prompt and response are one sequence; the model does not distinguish them.

Inside one layer

- Each token gets transformed into Q, K, V via three weight matrices.
- Attention: each token's Q dots against everyone's K, softmax, weight-mix the V's.
- Causal mask: each token only attends to itself and the past.

Why the cache works

- Causal mask means past tokens never look at the future.
- So past K and V never change once computed.
- Cache them, append new ones each step. Q is not cached.

Why inference is slow

- ~50 ms per token because we have to read 140 GB of weights from VRAM.
- The math itself is fast; the bottleneck is loading weights into compute.

Why speculative decoding works

- One forward pass costs the same whether we feed 1 new token or 4.
- A tiny model can guess 4 tokens cheaply.
- The big model verifies all 4 in one pass.
- A clever rejection rule makes the output match the big model's distribution exactly.
- Net effect: 2–3× speedup with no quality loss.

Vocabulary scoring

- Every prediction step computes scores for every vocabulary token.
- For Llama-3 that is 128,000 scores per token, every time.
- Sampling settings (temp, top-k, top-p) filter after this scoring.

End of guide. Read sections in order on first pass; jump to the glossary or cheat sheet for quick reference later.